Course : 23EC905 – EMBEDDED SYSTEM DESIGN USING IOT

Module : I – Fundamentals of Embedded System Design

Topic : Basics of Realtime Operating Systems

Operating System

What is an Operating System (1)?

- A modern computer consists of:
  - One or more processors
  - Main memory
  - Disks
  - Printers
  - Various input/output devices.

- Managing all these varied components requires a layer of software – the **Operating System (OS).**

What is an Operating System (2)?

- An Operating System is a program that acts as an intermediary/interface between a user of a computer and the computer hardware.

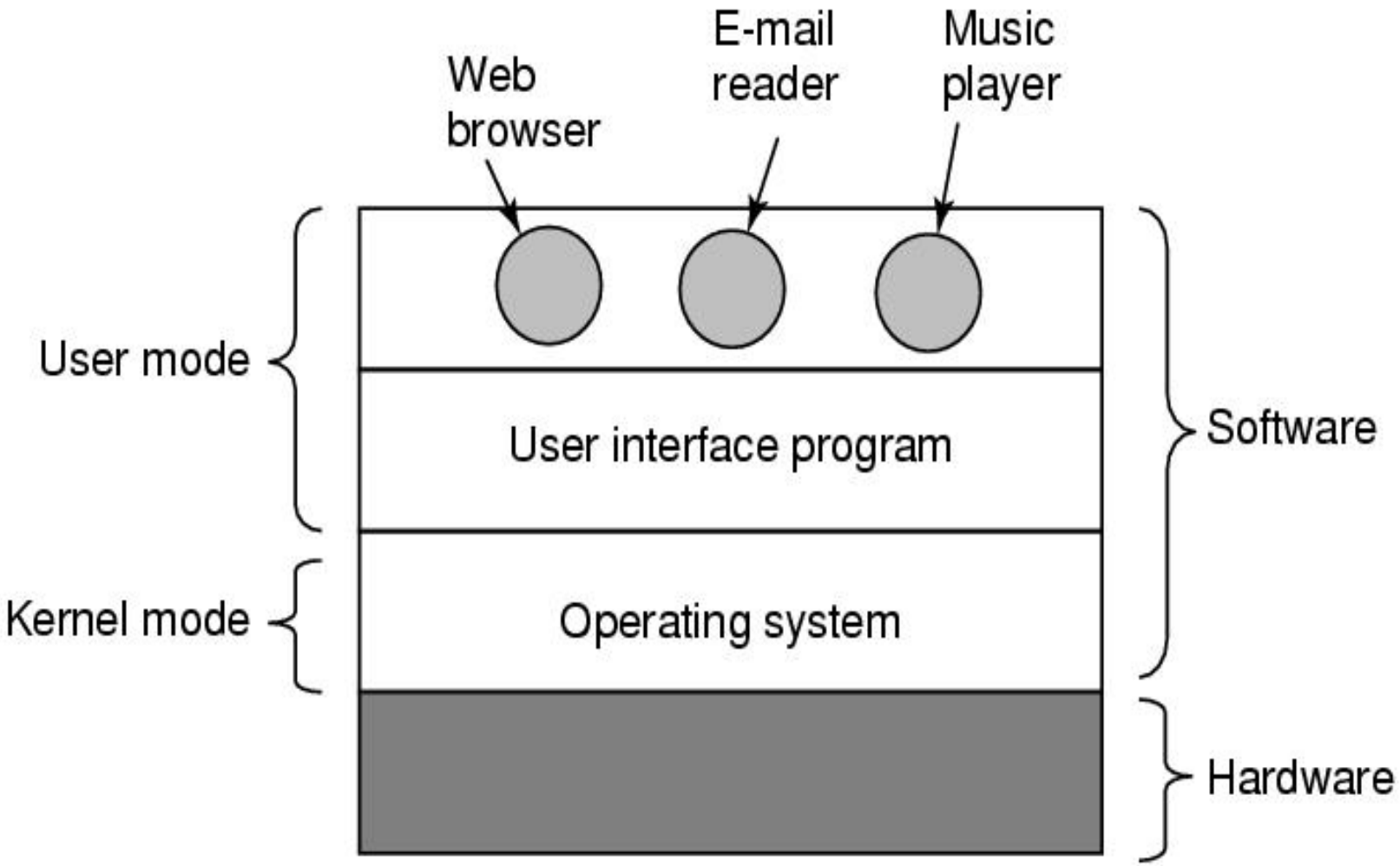
What is an Operating System?

- The operating system is what turns your hardware into a usable computer.
- The purpose of an operating system is to provide an environment in which a user can execute programs in a convenient and efficient manner.
- It allows user to communicate with devices and to perform desired functions.

Definition

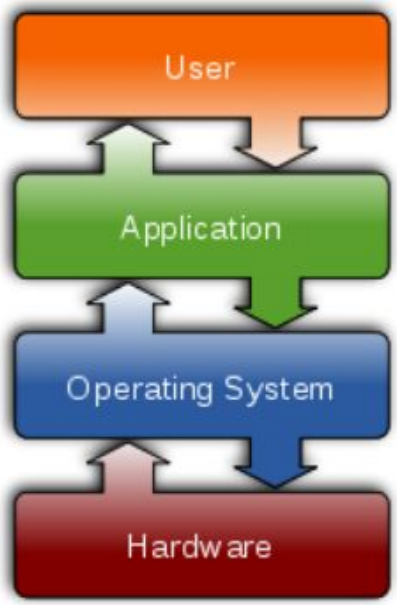
- An operating system is a program that controls the execution of application programs and acts as an interface between the user of a computer and the computer hardware.
- A more common definition is that the operating system is the one program running at all times on the computer (usually called the kernel), with all else being application programs.

Where does the OS fit in?



Operating systems

- **operating system**: Manages activities and resources of a computer.
  - software that acts as an interface between hardware and user
  - provides a layer of abstraction for application developers
- features provided by an operating system:
  - ability to execute programs (and multi-tasking)
  - memory management (and virtual memory)
  - file systems, disk and network access
  - an interface to communicate with hardware
  - a user interface (often graphical)
- **kernel**: The lowest-level core of an operating system.



Operating System

- Some popular Operating Systems include Linux , Windows , macOS, iOS, Android, etc. In the modern days, when speaking about servers, around 80% run GNU/Linux and 20% run Windows.

Operating System

- And talking about mobile devices, around 75% run Android (which uses the Linux kernel) and 25% run IOs (which is Apple's mobile OS).
- We can see here Linux is on top when we talk about servers, but regarding desktop segment Windows (with 73.72% market share) has more market share than Linux (with 3% market share)

Services provided by an OS

- Facilities for program creation
  - editors, compilers, linkers, debuggers, etc.
- Program execution
  - loading in memory, I/O and file initialization.
- Access to I/O and files
  - deals with the specifics of I/O and file formats.
- System access
  - resolves conflicts for resource contention.
  - protection in access to resources and data.

Why are Operating Systems Important?

- Important to understand and know how to correctly use when writing user applications.
- Large and complex systems that have a high economic impact and result in interesting problems of management.
- Few actually involved in OS design and implementation but nevertheless many general techniques to be learned and applied.
- Combines concepts from many other areas of Computer Science: Architecture, Languages, Data Structures, Algorithms, etc.

Computer System Components

1. Hardware – provides basic computing resources (CPU, Memory, I/O devices, Communication).
2. Operating System – controls and coordinates use of the hardware among various application programs for various users.
3. System & Application Programs – ways in which the system resources are used to solve computing problems of the users (Word processors, Compilers, Web browsers, Database systems, Video games).
4. Users – (People, Machines, other computers).

Design Goals

- User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast.
- System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient.

Operating Systems User Interfaces

Three levels of interface:

1. Graphics GUI (windows oriented)
2. Command level (also known as the shell). At login time, the shell starts computing
3. System calls invoked from user programs

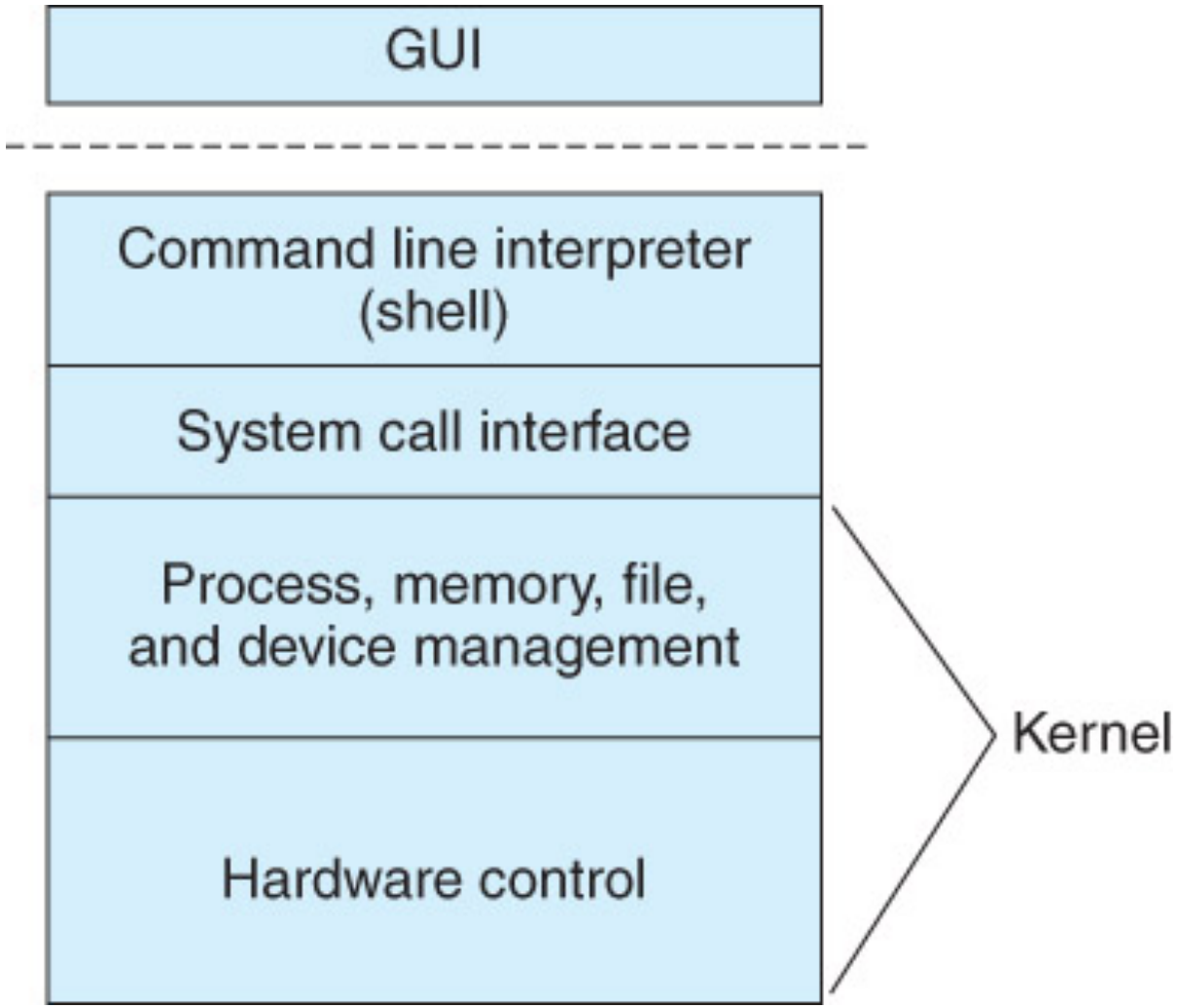
The Shell

- The program that handles user interaction with the system is called:
  - Shell
  - Command-line interpreter
- Two types of Shells
  - Graphical
  - Character oriented

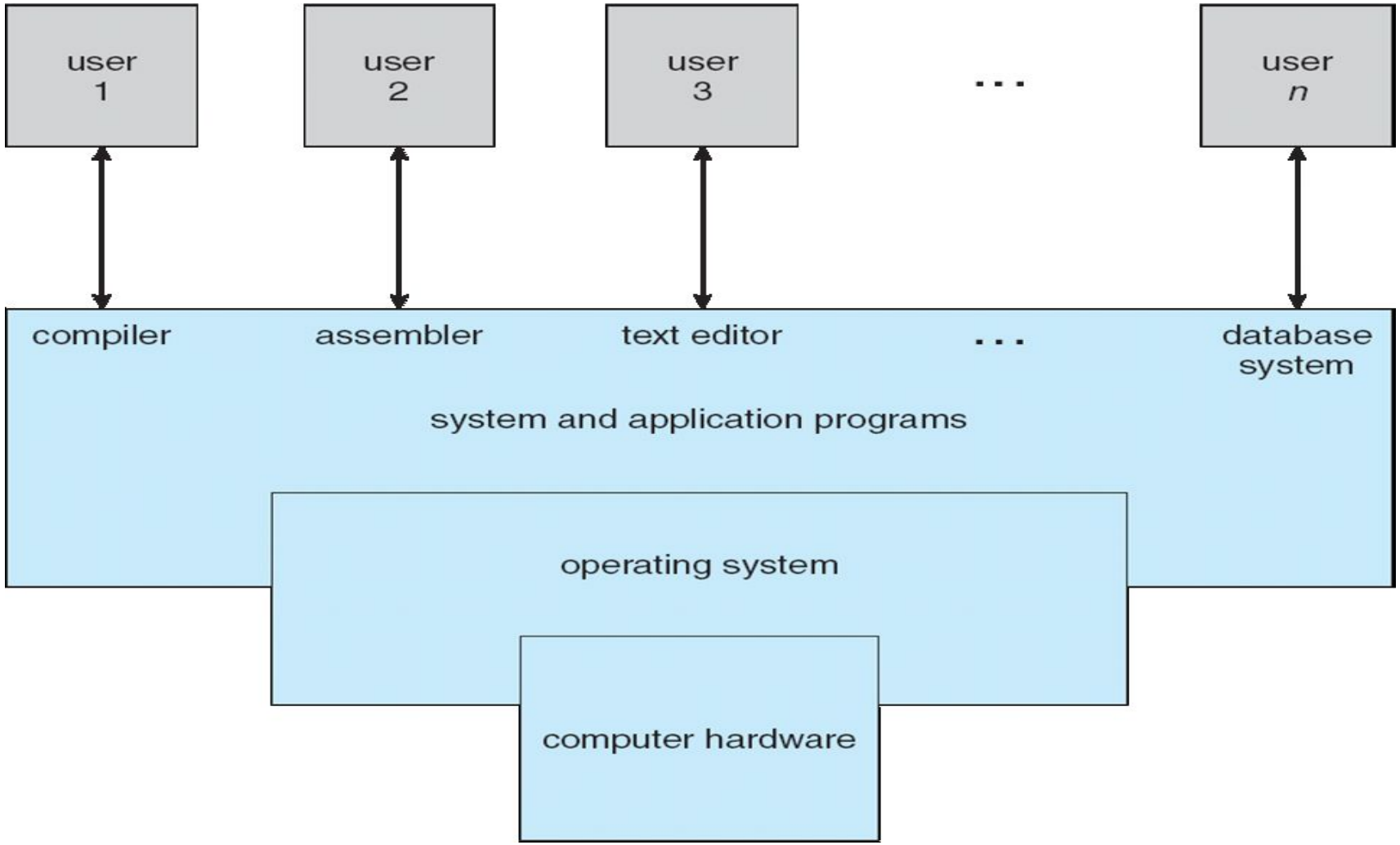
System Calls

- This is also known as the Application Programming Interface (API)
- Programs use the API to request the OS to perform some function

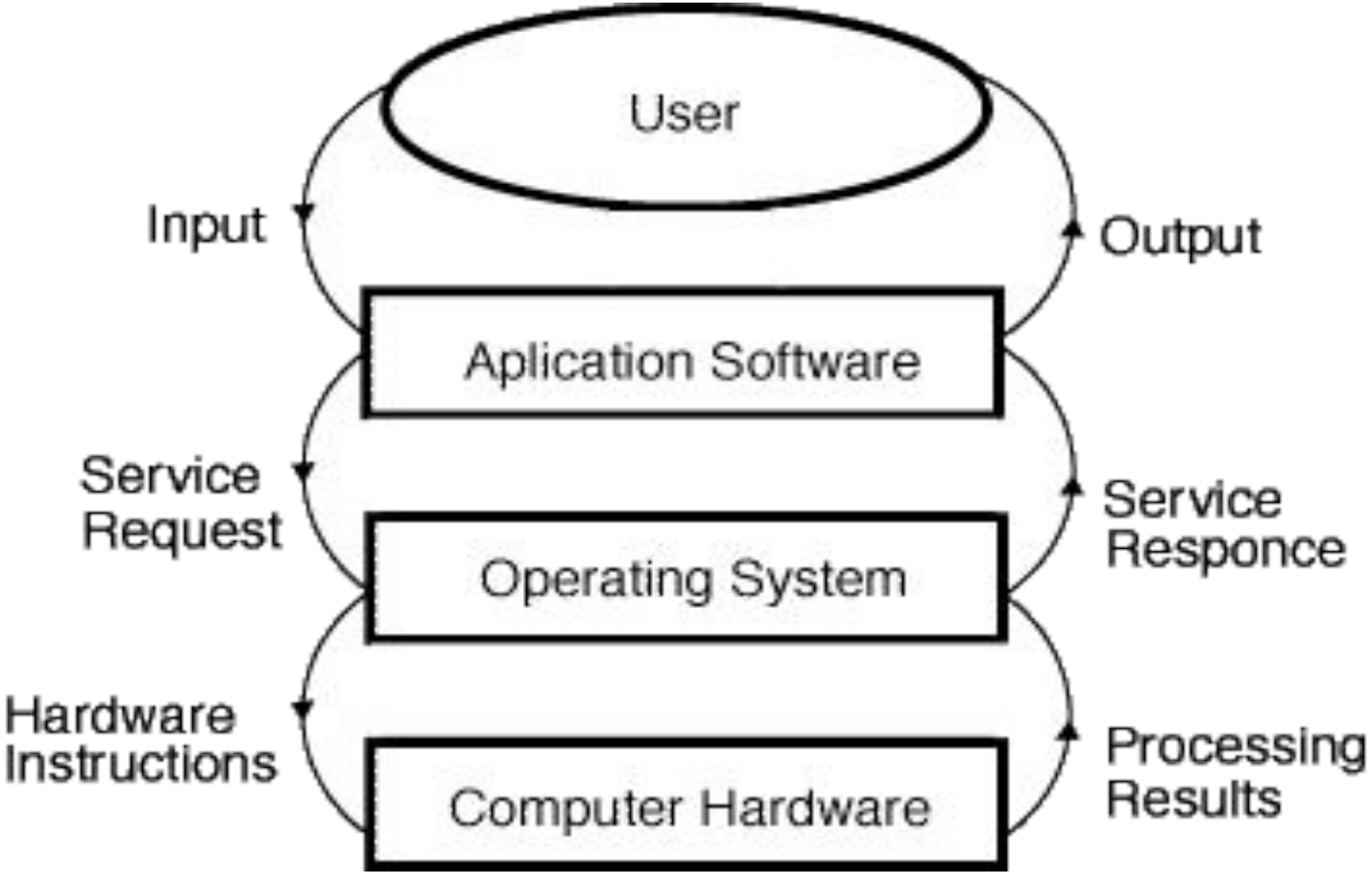
Basic Structure of an OS



Static View of System Components



Dynamic View of System Components



Functional Components of an OS

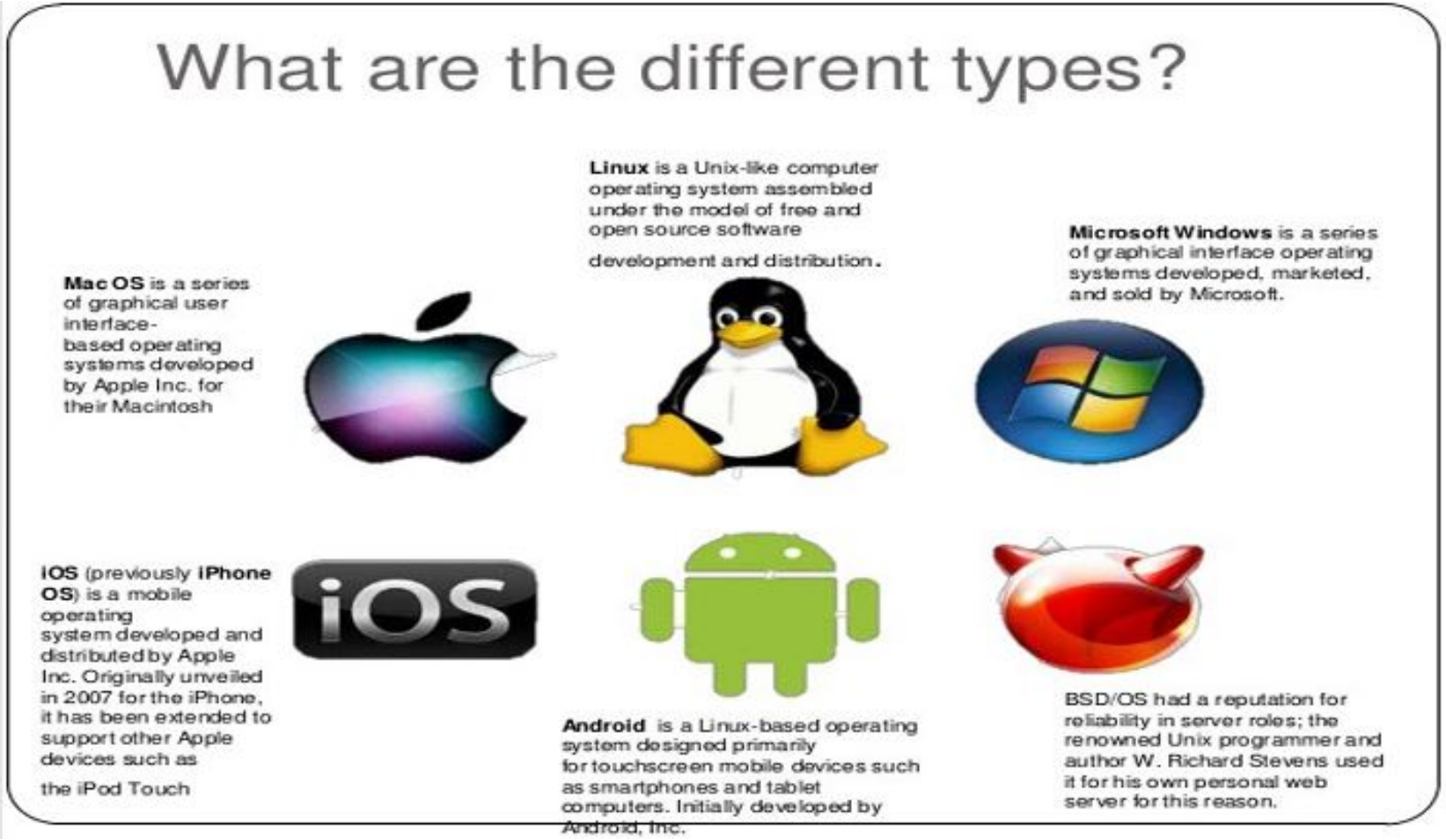
The most important components of an operating system are:

- Process manager
- Memory manager
- Resource manager
- File manager
- Device manager

Services Provided by the OS

- Process Control, execution, scheduling, etc.
- Communication between processes
- File Manipulation
- Device Manipulation
- Information Maintenance
- Memory Management

Operating systems



Types of Operating System

- **Batch Operating System**
- **Multiprogramming OS.**
- **Time-sharing or multitasking operating systems**
- **Distributed operating System**
- **Network operating System**
- **Real Time operating System**
  - **Hard real--time systems**
  - **Soft real--time systems**

Batch operating system

- This type of operating system does not interact with the computer directly.
- Batch processing is a technique in which an Operating System collects the programs and data together in a batch before processing starts.
- Each user prepares his job on an offline device like punch cards and submits it to the computer operator.
- To speed up processing, jobs with similar needs are batched together and run as a group. The programmers leave their programs with the operator and the operatorthen sorts programs with similar requirements into the batches

Batch operating system

The problems with Batch Systems are as follows

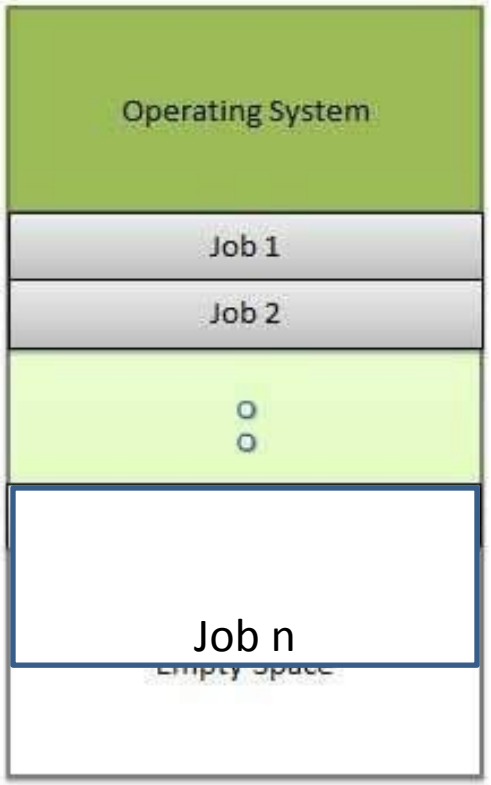
- Lack of interaction between the user and the job.
- CPU is often idle, because the speed of the mechanical I/O devices is slower than the CPU.
- Difficult to provide the desired priority.

Multiprogramming OS

- In such type of operating system, multiple jobs can sit simultaneously in the memory.
- For it, memory is considered as a partitioned device, so that each job can reside in a partition.
- Multiprogramming increases CPU utilization by organizing jobs so that the CPU always has one to execute.

Multiprogramming OS

The following figure shows the memory layout for a multiprogramming system.



Multiprogramming OS

- It takes one job from the memory and starts its execution.
- During execution eventually, the job requires some I/O operation so the job waits until I/O operation is complete.
- In non multiprogramming environment, during this time CPU sits idle. But in multiprogramming, the operating system switches to present job and executes another job.
- When that job waits for some task, CPU is switched to another job and so on.
- When first job finishes waiting, the CPU again starts its execution. So here the CPU time is fully utilized.

Multiprogramming OS

Advantages:

- Increase CPU utilization by eliminating idle time of CPU.
- Multiple jobs canexecute simultaneously.

Disadvantage:

- User cannot interact with his program during execution, i.e., it does not support interactivity.

Functions of Operating System in Multiprogramming

(a) Job Scheduling
(b) CPU scheduling
(c) Memory Management

(a) Job Scheduling- Since several jobs are ready in job pool to enter in the memory and if memory has not enough space to keep all the jobs, then it is necessary to choose some jobs out of all the jobs of job pool on some basis. This process of selecting jobs to enter in memory from the job pool is called job scheduling.

Functions of Operating System in Multiprogramming

(b) CPU Scheduling- If several jobs are ready in memory to run at the same time then it is necessary to choose a job to run. This Process of selecting job for execution out of all ready to run jobs in memory is called CPU scheduling.

© Memory Management- Since several jobs are loaded into memory simultaneously for execution, for this, memory management is required.

Time-sharing or Multitasking operating systems

- Time-sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time.
- Time-sharing or multitasking is a logical extension of multiprogramming.
- Processor's time which is shared among multiple users simultaneously is termed as time-sharing.
- The main difference between Multiprogrammed Batch Systems and Time-Sharing Systems is that in case of Multiprogrammed batch systems, the objective is to maximize processor use, whereas in Time-Sharing Systems,the objective is to minimize response time.

Time-sharing or Multitasking operating systems

- Multiple jobs are executed by the CPU, by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response.
- For example, in a transaction processing, the
- processor executes each user program in a short burst or quantum of computation. That is, if **n** users are present, then each user can get a time quantum.
- The operating system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time.
- Computer systems that were designed primarily as batch systems have been modified to time--sharing systems.

Time-sharing or Multitasking operating systems

Advantages of Timesharing operating systems are as follows −

- Provides the advantage of quick response.
- Avoids duplication of software.
- Reduces CPU idle time.

Time-sharing or Multitasking operating systems

Disadvantages of Time-

sharing operating systems are as follows

:

- Problem of reliability.
- Question of security and integrity of user programs and data.
- Problem of data communication.

Distributed operating System

- Distributed systems use multiple central processors to serve multiple real-time applications and multiple users.
- Data processing jobs are distributed among the processors accordingly.
- The processors communicate with one another through various communication lines (such as high-speed busses or telephone lines).
- These are referred as **loosely coupled systems or distributed systems.**
- Processors in a distributed system may vary in size and function.. These processors are referred as sites, nodes, computers,, and so on.

Distributed operating System

The advantages of distributed systems are as follows:

- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.
- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing.

Network operating System

- A Network Operating System runs on a server and provides the server the capability to manage data, user's, groups, security, applications, and other networking functions.
- The primary purpose of the network operating system is to allow shared file and printers access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.
- Examples of network operating systems include Microsoft Windows Server 2003 Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD.

Network operating System

The advantages of network operating systems are as follows –

- Centralized servers are highly stable.
- Security is server managed.
- Upgrades to new technologies and hardware can be easily integrated in to the system.
- Remote access to servers is possiblefrom different locations and types of systems.

Network operating System

- The disadvantages of networks operating systems are as follows –
- High cost of buying and running a server.
- Dependency on a central location for most operations.
- Regular maintenance and updates are required.

Real Time operating System

- A real--time system is defined as a data processing system in which the time interval required to processes and respond to inputs is so small that it controls the environment.
- The time taken by the system to respond to an input and display of required updated information is termed as the **response time. So in** this method, the response time is very less as compared to online processing.
- Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application.
- A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail. For example, Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.
- There are two types of real-time operating systems.

Hard real--time systems

- Hard real--time systems guarantee that critical tasks complete on time.
- In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM.
- In these systems, virtual memory is almost never found.

Soft real-time systems

- Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes.
- Soft real-time systems have limited utility than hard real-time systems.
- For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

Mainframe OS

- The mainframe OS are designed to handle large amount of disk space and large number of I/O devices.
- Heavily oriented toward processing many jobs at once.
- They typically provides three sort of services batch, transaction processing, and time sharing.
- The file protection is necessary, since many users handle these types of computers.
- These OS are used for high-end Web servers, for business-to-business transaction etc.

Personal Computer Operating Systems

- Designed to handle small amount of disk space and small number of I/O devices.
- Some personal computer operating system handle only one job at a time, like MS-DOS.
- They typically provide a good interface to a single user.
- The file protection is not necessary in some personal computer but if these PCs are used on network then file protection becomes a necessary feature for such operating systems.
- These operating systems are used for personal computer those perform computer those perform word processing, prepare spreadsheets, and access internet.
- Example- DOS, Windows.., Macintosh, Linux, etc.

Multiprocessor OS

- The multiprocessors system need special types of operating systems.
- The main feature of such operating system is that they allow many processes to run simultaneously without a significant lack of performance.
- These also provide a better way to ensure, that in I/O operation data is coming and reaching to the appropriate CPU.
- Example – Windows NT, Solaris, Linux etc.

Scheduling Process

- Scheduling is the process of deciding which task should be executed at any point in time based on a predefined algorithm.
- The logic for the scheduling is implemented in a functional unit called the scheduler.

SCHEDULER

- A scheduler provides the algorithms needed to determine which task executes when. It includes
  - Schedulable entities
  - Multitasking
  - Context switching
  - Dispatcher
  - Scheduling algorithms

Scheduler Algortihms

- In real-time systems, the scheduler is considered as the most important component which is typically a short-term task scheduler.
- The main focus of this scheduler is to reduce the response time associated with each of the associated processes instead of handling the deadline.

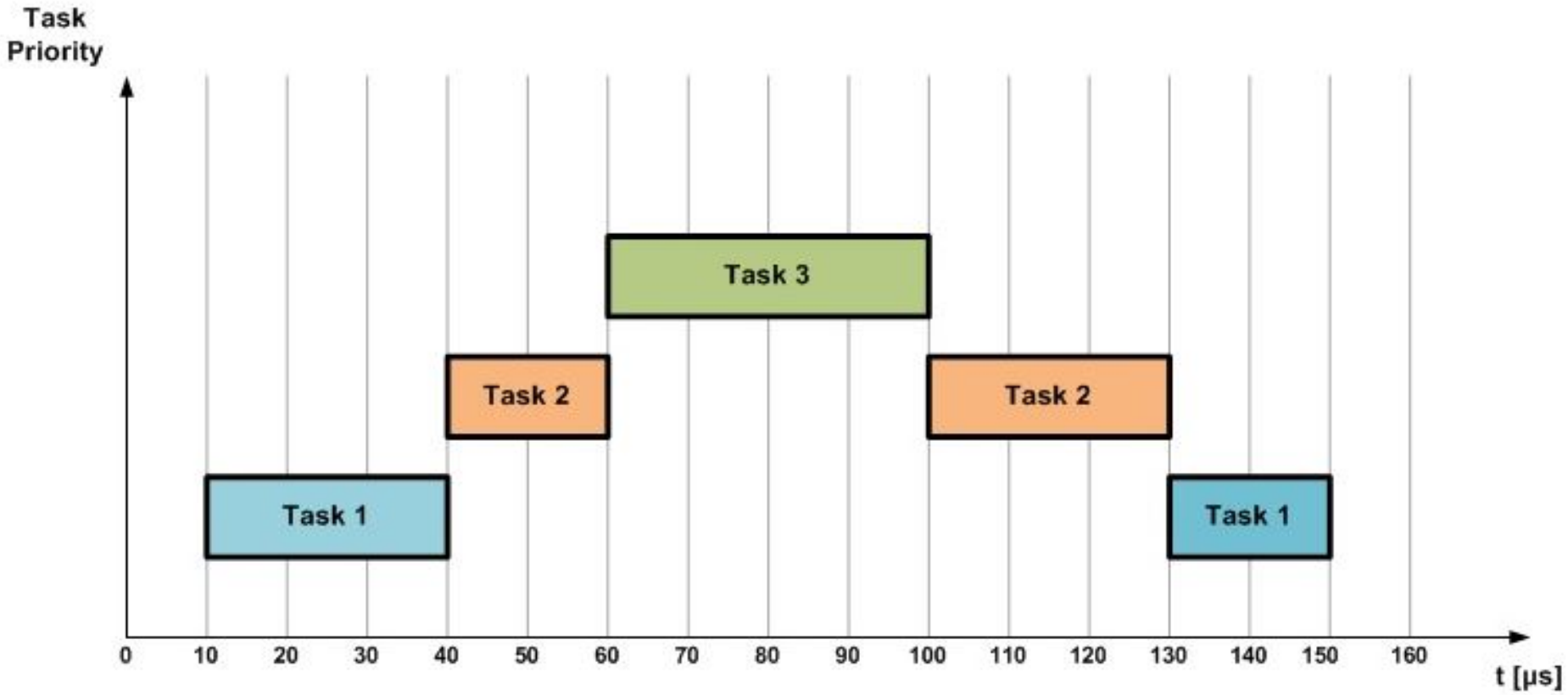
Types of Scheduling Algorithms

- There are many scheduling algorithms that can be used for scheduling task execution on a CPU.
- They can be classified into two main types: preemptive scheduling algorithms and non-preemptive scheduling algorithms.

Preemptive Scheduling

- Preemptive scheduling allows the interruption of a currently running task, so another one with more “urgent” status can be run.
- The interrupted task is involuntarily moved by the scheduler from running state to ready state. This dynamic switching between tasks that this algorithm employs is, in fact, a form of multitasking.
- It requires assigning a priority level for each task. A running task can be interrupted if a task with a higher priority enters the queue.

Preemptive Scheduling



Preemptive Scheduling

Task Name	**Arrival Time** [μs]	**Execute Time** [μs]
Task 1	10	50
Task 2	40	50
Task 3	60	40

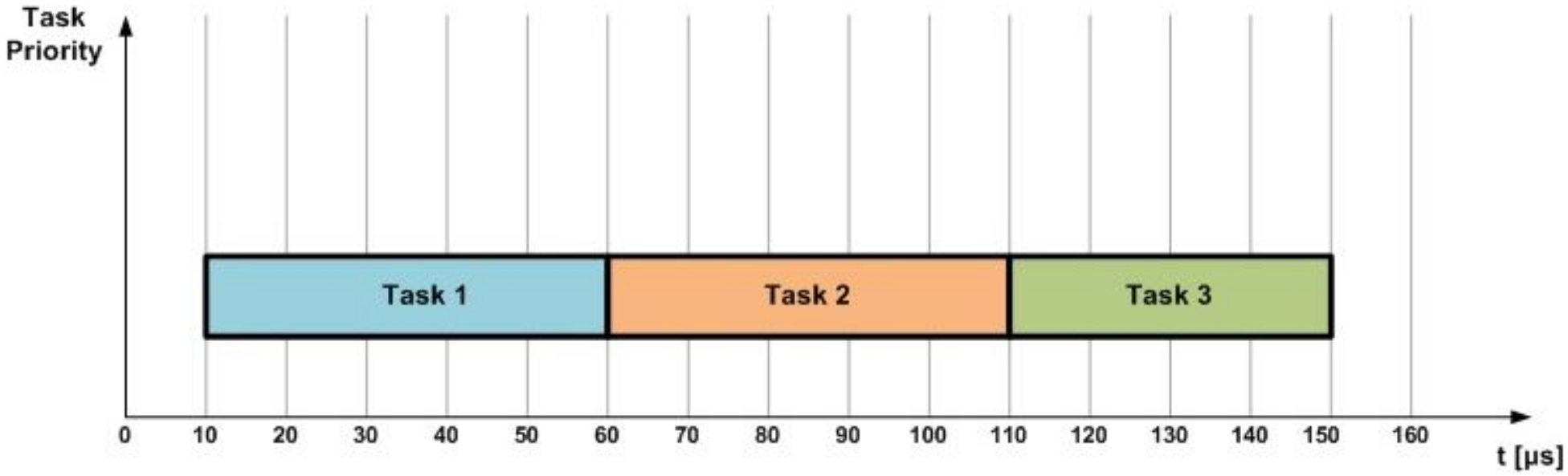
Preemptive Scheduling

- In the above fig, we can see that Task 1 is the first to start executing, as it is the first one to arrive (at t = 10 μs).
- Task 2 arrives at t = 40μs and since it has a higher priority, the scheduler interrupts the execution of Task 1 and puts Task 2 into running state.
- Task 3 which has the highest priority arrives at t = 60 μs. At this moment Task 2 is interrupted and Task 3 is put into running state. As it is the highest priority task it runs until it completes at t = 100 μs.
- Then Task 2 resumes its operation as the current highest priority task. Task 1 is the last to complete is operation.

Non-preemptive Scheduling (a.k.a Co-Operative Scheduling)

- In non-preemptive scheduling, the scheduler has more restricted control over the tasks. It can only start a task and then it has to wait for the task to finish or for the task to voluntarily return the control.
- A running task can’t be stopped by the scheduler.

Non-preemptive Scheduling



Non-preemptive Scheduling

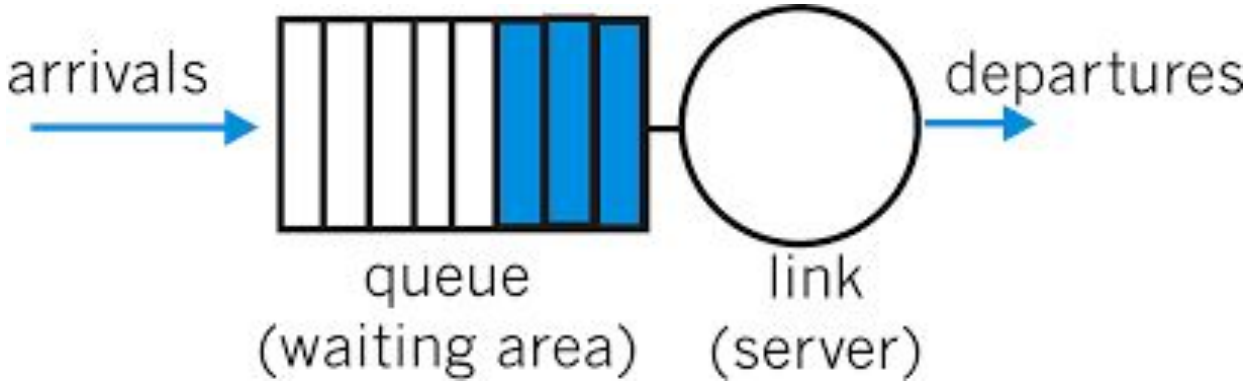
- If we take the three tasks specified in the table from the previous chapter and schedule them using a non-preemptive algorithm we get the behavior shown in the above fig.
- Once started, each task completes its operation and then the next one starts.
- The non-preemptive scheduling can simplify the synchronization of the tasks, but that is at the cost of increased response times to events. This reduces its practical use in complex real-time systems.

POPULAR SCHEDULING ALGORITHMS

- First In First Out
- Round-robin Algorithm
- Round robin with priority
- Shortest Job first
- Non-preemptive multitasking
- Preemptive multitasking

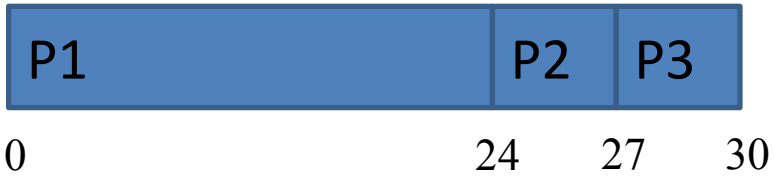
FIRST IN FIRST OUT (FIFO)

- In FIFO, the tasks which are Ready-to-Run are kept in a queue and the CPU serves the tasks on FCFS basis.
- Very simple – not well suited for many applications – difficult to estimate the amount of a time a task has to wait for being executed.
- Good for small tasks with small execution time.



SCHEDULING ALGORITHMS: FIRST-COME, FIRST-SERVED (FCFS)

- Example: Three processes arrive in order P1, P2, P3.
  - P1 burst time: 24
  - P2 burst time: 3
  - P3 burst time: 3
- Waiting Time
  - P1: 0
  - P2: 24
  - P3: 27
- Completion Time:
  - P1: 24
  - P2: 27
  - P3: 30
- Average Waiting Time: (0+24+27)/3 = 17
- Average Completion Time: (24+27+30)/3 = 27

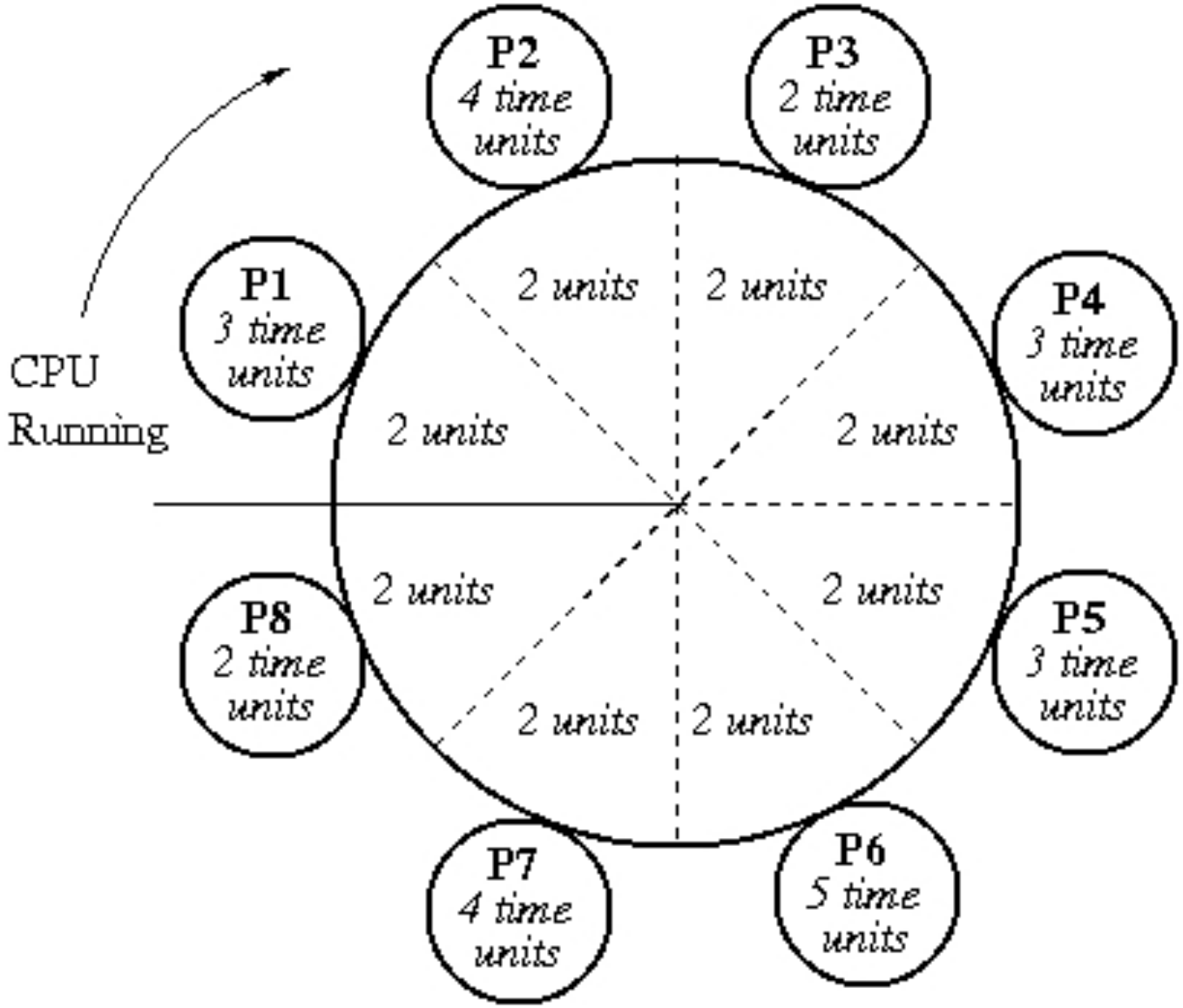


SCHEDULING ALGORITHMS: FIRST-COME, FIRST-SERVED (FCFS)

- What if their order had been P2, P3, P1?
  - P1 burst time: 24
  - P2 burst time: 3
  - P3 burst time: 3

ROUND ROBIN SCHEDULER

- Appropriate for time-sharing environments
- The kernel allocates a certain amount of time for each task waiting in the queue.
- The time slice allocated to each task is called quantum.
- If n processes are running under round robin they will have the illusion they have exclusive use of a processor running at 1/n times the actual processor speed


P2
4 time units
P3
2 time units
P1
3 time units
P4
3 time units
CPU Running
2 units
2 units
2 units
2 units
2 units
2 units
2 units
2 units
P8
2 time units
P5
3 time units
P7
4 time units
P6
5 time units

ROUND ROBIN (RR) SCHEDULING

- The kernel gives the control to the next task if

  The current task has completed its work within the time slice

  the current task has no work to do

  the current task has completed its allocated time slice

- It is very simple to implement, but there are no priorities of any task.

- All tasks are considered of equal importance.

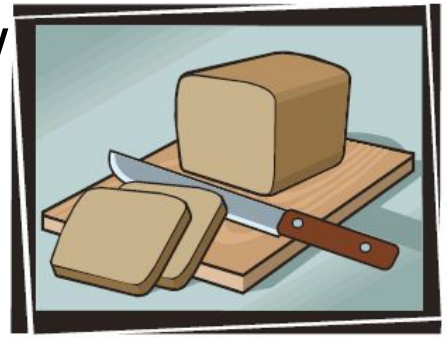
- Network packet scheduling use this Scheduling algorithm

ROUND ROBIN WITH PRIORITY

- RR can be slightly modified by assigning priority levels to some or all the tasks.
- A high priority task can interrupt the CPU so that it can be executed.
- This scheduling algorithm can meet the desired response time for a high priority task.
- Soft real time systems can use this algorithm.
- Ex : Interactive applications

RR SUMMARY

- Pros and Cons:
  - Better for short jobs (+)
  - Fair (+)
  - Context-switching time adds up for long jobs (-)
    - The previous examples assumed no additional time was needed for context switching – in reality, this would add to wait and completion time without actually progressing a process tow n.
    - Remember: the OS consumes resources, too!



- If the chosen quantum is
  - too large, response time suffers
  - infinite, performance is the same as FIFO
  - too small, throughput suffers and percentage overhead grows.

RR SUMMARY

- Actual choices of time-slice:
  - UNIX: initially 1 second:
    - Worked when only 1-2 users
    - If there were 3 compilations going on, it took 3 seconds to echo each keystroke.
    - In practice, need to balance short-job performance and long-job throughput.
    - Typical time-slice 10ms-100ms
    - Typical context-switch overhead 0.1ms – 1ms (about 1%)

SHORTEST JOB FIRST (SJF)

- If the time for each task can be estimated beforehand, the CPU can execute the task which takes the least amount of time.

- It is like a priority assignment, the priority is being decided by the amount of time – the higher the execution time, the lesser the priority.

- Advantage – more number of tasks will be executed.

- Not suitable for hard real time systems.

NON-PREEMPTIVE SCHEDULING ALGORITHMS

- Non-preemptive means that once a process is running it will continue to do so until it relinquishes control of the CPU.
- This would be because it terminates, voluntarily yields the CPU to some other process (waits) or requests some service from the operating system.
- This is non preemptive multitasking, also known as cooperative multitasking.
- Each task has to release the CPU and give control to another task on its own.
- Each task is given a priority, but the priority to be respected by the other tasks.

PREEMPTIVE SCHEDULING ALGORITHMS

- In preemptive multitasking, the higher priority task is always given the CPU time.
- If a low priority task is presently running and a higher priority task is in Ready-to-Run state, the running task is preempted and the higher priority task is executed.
- Most of the commercial embedded operating systems use preemptive multitasking to meet real time requirements.

RATE MONOTONIC ANALYSIS (RMA)

- Priority assignment to a task can be Static or Dynamic.

- While designing an embedded system
  - Need to take a list of task
  - Assign priority to each task

- RMA may not be accurate for any application, but it is a good starting point.

RMA

- RMA makes the following assumptions
- High priority task will run first.
- All the tasks run at a regular intervals
- Tasks do not synchronize with each other

- In RMA, the priority is proportional to the frequency of execution.

- E_i = Execution Time, T_i = Execution period, of task i

- Schedulability test – How much CPU test is utilized by the tasks.

RMA

- When you are designing an embedded system, it will be a good idea to start with RMA.
- List all the tasks, estimate the execution period and execution time.
- As a thumb rule, ensure that the CPU utilization is not above 70%.
- Assign priorities based on execution frequency.

INTERRUPT SERVICE ROUTINE

- ✔ Interrupt is a hardware signal that informs CPU that an important event has occurred.
- ✔ When interrupt occurs, CPU save its context and jumps to the ISR.
- ✔ After processes the event, CPU returns to the interrupted task in a non-preemptive kernel.
- ✔ Interrupt Latency : The maximum time for which interrupts are disabled + time to start the execution of the first instruction in the ISR is called interrupt latency.

ISR

- Interrupt Response time :

Time between receipt of interrupt signal and starting the code that handles the interrupt is called interrupt response time.

- Interrupt recovery time :

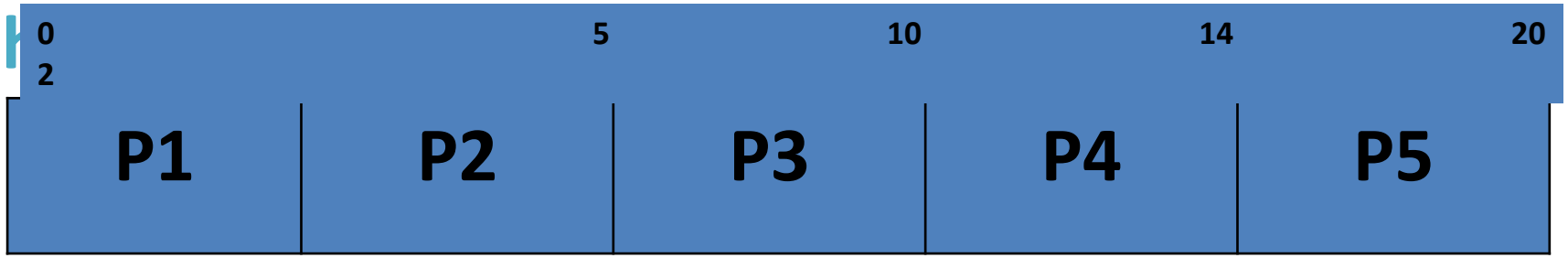
Time required for CPU to return to the interrupted code/highest priority task is called interrupt recovery time.

FCFS

Find the Execution order, Completion time, Turnaround Time and Waiting time

Process	Arrival Time	Burst Time
P1	0	2
P2	1	3
P3	2	5
P4	3	4
P5	4	6

Gantt Ch



- Turn Around Time: Exit/Completion time – Arrival Time
- Waiting Time: Turn Around Time – Burst Time

Process	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	0	2			
P2	1	3			
P3	2	5			
P4	3	4			
P5	4	6			

- Average Turn Around Time =
- Average Waiting Time =

Process	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	0	2	2	2	0
P2	1	3	5	4	1
P3	2	5	10	8	3
P4	3	4	14	11	7
P5	4	6	20	16	10

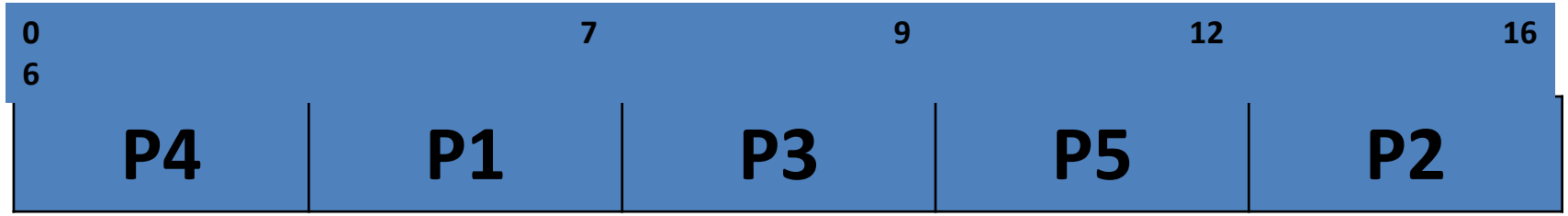
- Average Turn Around Time = 8.2
- Average Waiting Time = 4.2

Shortest Job First – Non Preemptive

Find the Execution order, Completion time, Turnaround Time and Waiting time

Process	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Gantt Chart



Process	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	3	1	7	4	3
P2	1	4	16	15	11
P3	4	2	9	5	3
P4	0	6	6	6	0
P5	2	3	12	10	7

- Average Turn Around Time = 8
- Average Waiting Time = 4.8

Shortest Job First – Preemptive

Find the Execution order, Completion time, Turnaround Time and Waiting time

Process	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Shortest Job First – Preemptive

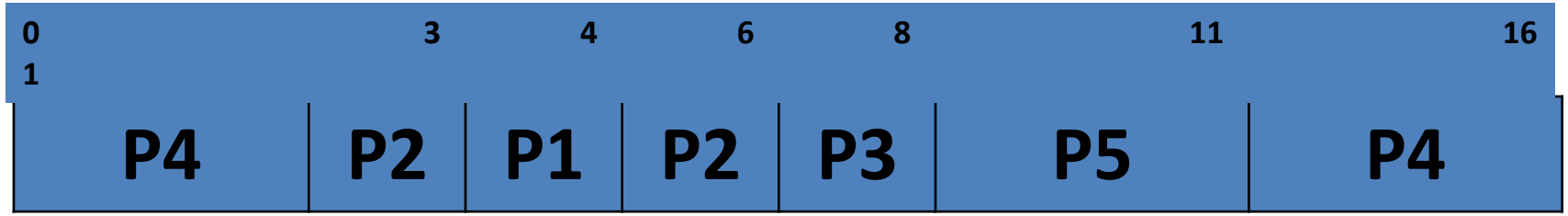
Find the Execution order, Completion time, Turnaround Time and Waiting time

Process	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Gantt Chart

0	1	3	4	6	8	11	16
P4	P2	P1	P2	P3	P5	P4	

Gantt Chart



Process	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	3	1	4	1	0
P2	1	4	6	5	1
P3	4	2	8	4	2
P4	0	6	16	16	10
P5	2	3	11	9	6

- Average Turn Around Time = 7
- Average Waiting Time = 3.8

MULTIPLE TASKS

Tasks and Processes:

- Many (if not most) embedded computing systems do more than one thing that is, the environment can cause mode changes that in turn cause the embedded system to behave quite differently.

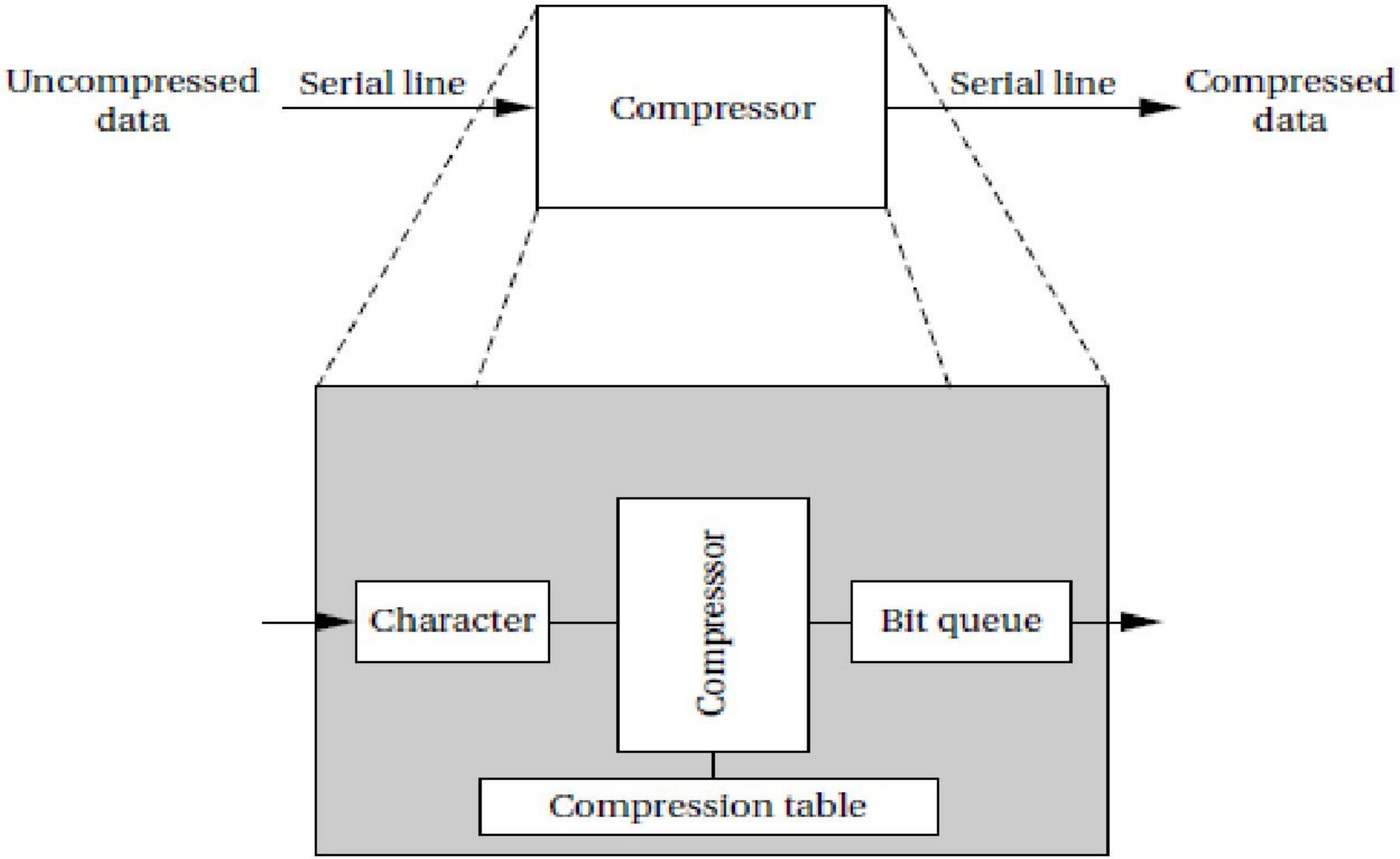
- For example, when designing a telephone answering machine, we can define recording a phone call and operating the user's control panel as distinct tasks, because they perform logically distinct operations and they must be performed at very different rates.

- These different **tasks** are part of the system's functionality, but that application-level organization of functionality is often reflected in the structure of the program as well.

PROCESS

- A ***process*** is a single execution of a program.
- If we run the same program two different times, we have created two different processes.
- Each process has its own state that includes not only its registers but all of its memory.
- In some OSs, the memory management unit is used to keep each process in a separate address space.
- In others, particularly lightweight RTOSs, the processes run in the same address space.
- Processes that share the same address space are often called ***threads.***

FLY COMPRESSION BOX



FLY COMPRESSION BOX

- As shown, this device is connected to serial ports on both ends.
- The input to the box is an uncompressed stream of bytes.
- The box emits a compressed string of bits on the output serial line, based on a predefined compression table.
- Such a box may be used, for example, to compress data being sent to a modem.

FLY COMPRESSION BOX

- The program's need to receive and send data at different rates for example, the program may emit 2 bits for the first byte and then 7 bits for the second byte will obviously find itself reflected in the structure of the code.

- It is easy to create irregular, ungainly code to solve this problem; a more elegant solution is to create a queue of output bits, with those bits being removed from the queue and sent to the serial port in 8-bit sets.

- But beyond the need to create a clean data structure that simplifies the control structure of the code, we must also ensure that we process the inputs and outputs at the proper rates.

- For example, if we spend too much time in packaging and emitting output characters, we may drop an input character.

- Solving timing problems is a more challenging problem.

FLY COMPRESSION BOX

The text compression box provides a simple example of rate control problems.

A control panel on a machine provides an example of a different type of rate control problem, the ***asynchronous input***.

The control panel of the compression box may, for example, include a compression mode button that disables or enables compression, so that the input text is passed through unchanged when compression is disabled.

We certainly do not know when the user will push the compression mode button the button may be depressed asynchronously relative to the arrival of characters for compression.

MULTI-RATE SYSTEMS

- Implementing code that satisfies timing requirements is even more complex when multiple rates of computation must be handled.

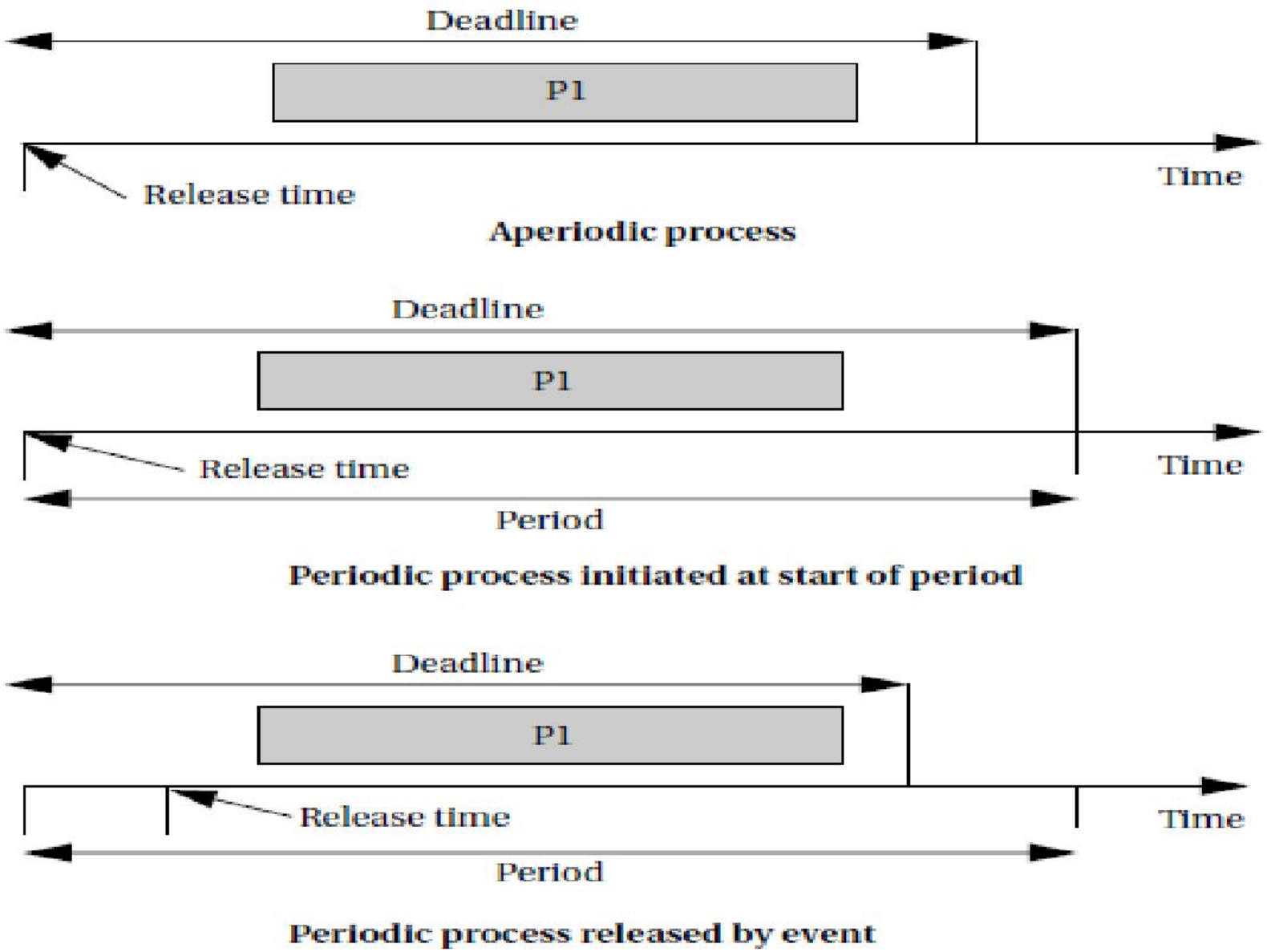
- ***Multi-rate*** embedded computing systems are very common, including automobile engines, printers, and cell phones.

- In all these systems, certain operations must be executed periodically, and each operation is executed at its own rate.

TIMING REQUIREMENTS ON PROCESSES

- Processes can have several different types of timing requirements imposed on them by the application.
- The timing requirements on a set of processes strongly influence the type of scheduling that is appropriate.
- A scheduling policy must define the timing requirements that it uses to determine whether a schedule is valid.
- Before studying scheduling proper, we outline the types of process timing requirements that are useful in embedded system design.

EXAMPLE DEFINITIONS OF RELEASE TIMES AND DEADLINES



EXAMPLE DEFINITIONS OF RELEASE TIMES AND DEADLINES

- It illustrates different ways in which we can define two important requirements on processes: ***release time*** and ***deadline***.

- The release time is the time at which the process becomes ready to execute; this is not necessarily the time at which it actually takes control of the CPU and starts to run.

- An aperiodic process is by definition initiated by an event, such as external data arriving or data computed by another process.

- The release time is generally measured from that event, although the system may want to make the process ready at some interval after the event itself.

- For a periodically executed process, there are two common possibilities.
- In simpler systems, the process may become ready at the beginning of the period.
- More sophisticated systems, such as those with data dependencies between processes, may set the release time at the arrival time of certain data, at a time after the start of the period.
- A deadline specifies when a computation must be finished.
- The deadline for an aperiodic process is generally measured from the release time, since that is the only reasonable time reference.

- The deadline for a periodic process may in general occur at some time other than the end of the period.

- Rate requirements are also fairly common. A rate requirement specifies how quickly processes must be initiated.

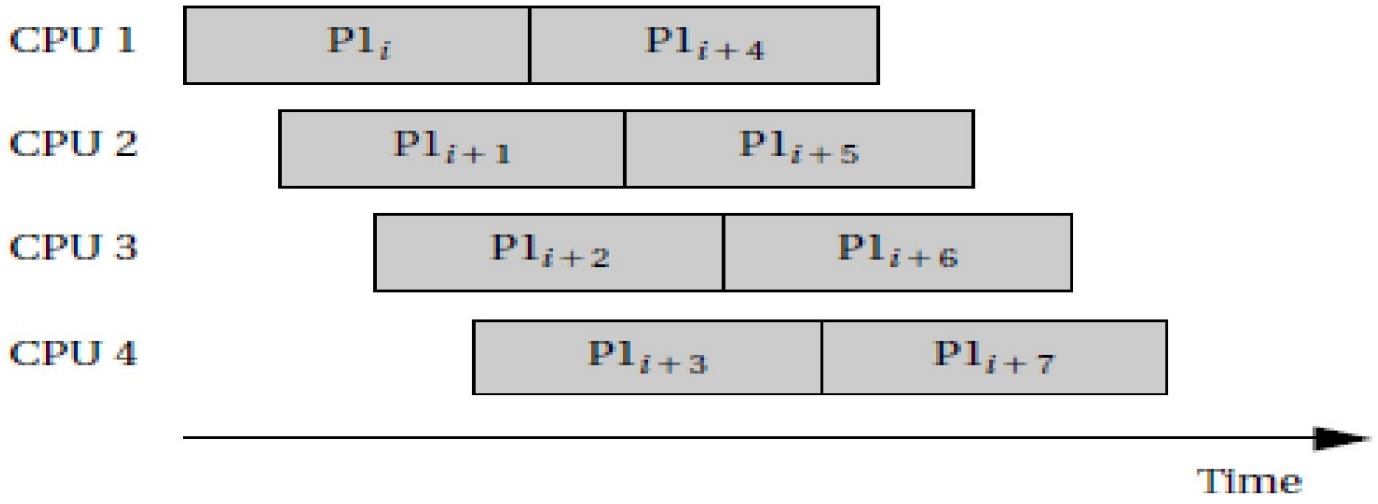
- The ***period*** of a process is the time between successive executions.

- For example, the period of a digital filter is defined by the time interval between successive input samples.

- The process's ***rate*** is the inverse of its period.

- In a multirate system, each process executes at its own distinct rate.

A SEQUENCE OF PROCESSES WITH A HIGH INITIATION RATE



- The most common case for periodic processes is for the initiation interval to be equal to the period.
- However, pipelined execution of processes allows the initiation interval to be less than the period.
- It illustrates process execution in a system with four CPUs.

CPU Metrics

- We also need some terminology to describe how the process actually executes.
- The ***initiation time*** is the time at which a process actually starts executing on the CPU.
- The ***completion time*** is the time at which the process finishes its work.
- The most basic measure of work is the amount of ***CPU time*** expended by a process.
- The CPU time of process *i* is called C_i .
- Note that the CPU time is not equal to the completion time minus initiation time; several other processes may interrupt execution.
- The total CPU time consumed by a set of processes is $T = \Sigma T_i$

Utilization

- We need a basic measure of the efficiency with which we use the CPU.

- The simplest and most direct measure is ***utilization***.

- *U* = (CPU time for useful work)/(total available CPU time).

- Utilization is the ratio of the CPU time that is being used for useful computations to the total available CPU time.

- This ratio ranges between 0 and 1, with 1 meaning that all of the available CPU time is being used for system purposes.

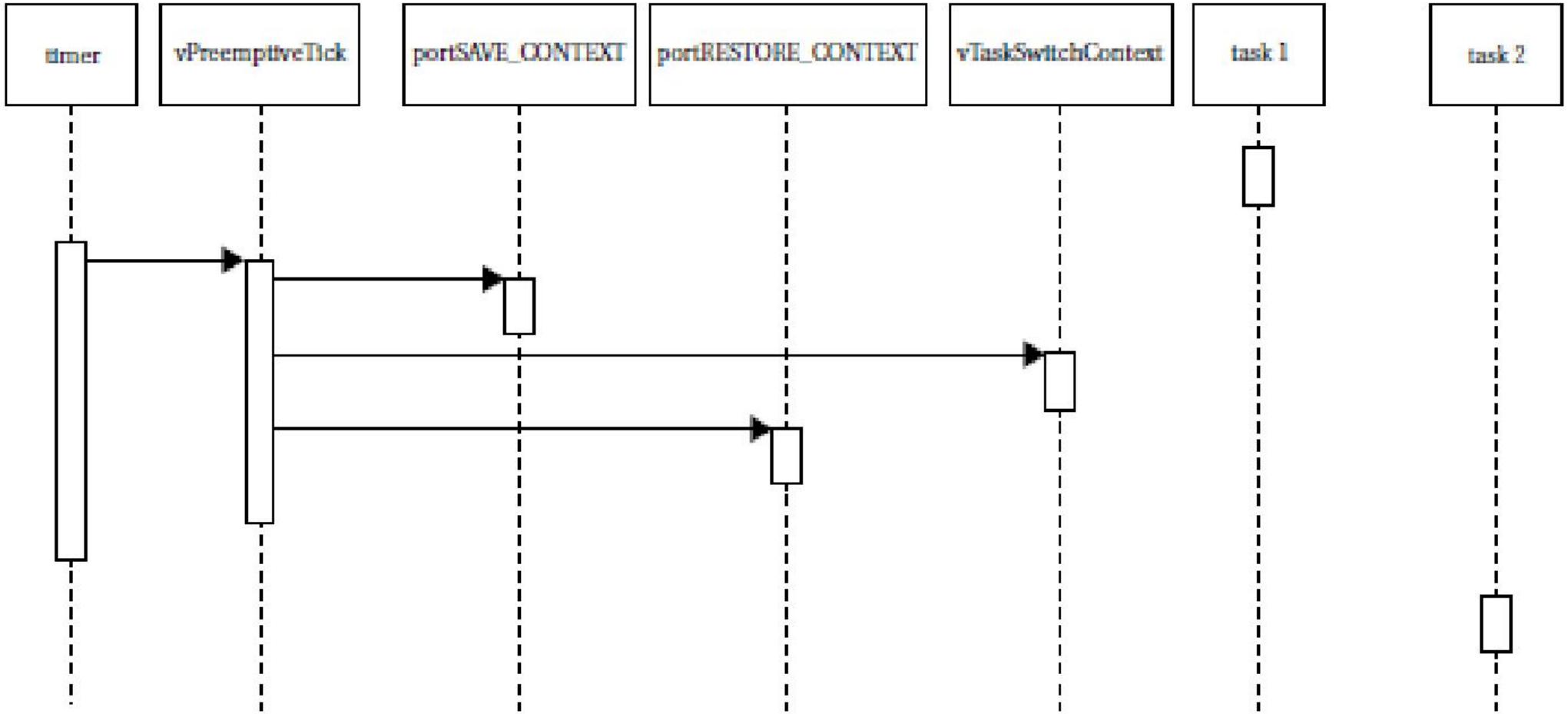
- The utilization is often expressed as a percentage.

- If we measure the total execution time of all processes over an interval of time *t*, then the CPU utilization is $U=T/t$.

PROCESSES AND CONTEXT SWITCHING

- The best way to understand processes and context is to dive into an RTOS implementation.
- We will use the FreeRTOS.org kernel as an example; in particular, we will use version 4.7.0 for the ARM7 AT91 platform.
- A process is known in FreeRTOS.org as a task.
- Task priorities in FreeRTOS.org are ranked opposite to the convention we use in the rest of the book: higher numbers denote higher priorities and the priority 0 task is the idle task.

SEQUENCE DIAGRAM FOR FREERTOS.ORG CONTEXT SWITCH



- To understand the basics of a context switch, let's assume that the set of tasks is in steady state: Everything has been initialized, the OS is running, and we are ready for a timer interrupt.

- Sequence diagram for a context switch in freeRTOS.org.

- This diagram shows the application tasks, the hardware timer and all the functions in the kernel that are involved in the context switch:

- vPreemptiveTick () is called when the timer ticks.

- portSAVE_CONTEXT() swaps out the current task context.

- vTaskSwitchContext ( ) chooses a new task.

- portRESTORE_CONTEXT() swaps in the new context.

OPERATING SYSTEMS

- An Operating system is a program that controls the execution of application programs and acts as an interface between the user of a computer and the computer hardware.

- A more common definition is that the operating system is the one program running at all times on the computer (usually called the kernel), with all else being applications programs.

- An Operating system is concerned with the allocation of resources and services, such as memory, processors, devices and information.

- The Operating System correspondingly includes programs to manage these resources, such as a traffic controller, a scheduler, memory management module, I/O programs, and a file system.

SCHDULING POLICIES

- A ***scheduling policy*** defines how processes are selected for promotion from the ready state to the running state.

- Every multitasking OS implements some type of scheduling policy.

- Choosing the right scheduling policy not only ensures that the system will meet all its timing requirements, but it also has a profound influence on the CPU horsepower required to implement the system's functionality.

- Schedulability means whether there exists a schedule of execution for the processes in a system that satisfies all their timing requirements.

- In general, we must construct a schedule to show schedulability, but in some cases we can eliminate some sets of processes as unschedulable using some very simple tests.

Utilization is one of the key metrics in evaluating a scheduling policy.

Our most basic requirement is that CPU utilization be no more than 100% since we can't use the CPU more than 100% of the time.

When we evaluate the utilization of the CPU, we generally do so over a finite period that covers all possible combinations of process executions.

For periodic processes, the length of time that must be considered is the ***hyperperiod***, which is the least-common multiple of the periods of all the processes.

The complete schedule for the least-common multiple of the periods is sometimes called the ***unrolled schedule***.

If we evaluate the hyperperiod, we are sure to have considered all possible combinations of the periodic processes.

CYCLOSTATIC SCHEDULING

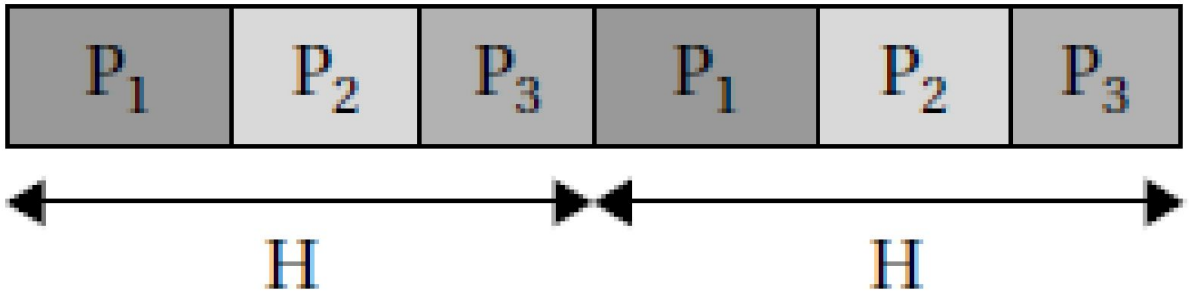
- We will see that some types of timing requirements for a set of processes imply that we cannot utilize 100% of the CPU's execution time on useful work, even ignoring context switching overhead.

- However, some scheduling policies can deliver higher CPU utilizations than others, even for the same timing requirements.

- One very simple scheduling policy is known as **cyclostatic** scheduling or sometimes as **Time Division Multiple Access** scheduling.

- A cyclostatic schedule is divided into equal-sized time slots over an interval equal to the length of the hyperperiod *H*.

- Processes always run in the same time slot.



ROUND-ROBIN SCHEDULING

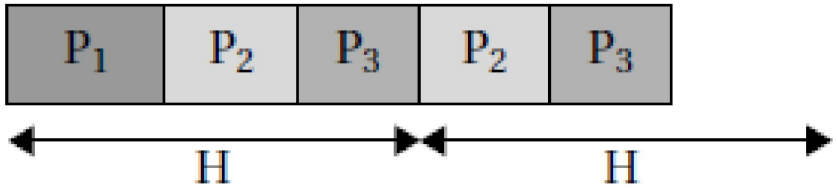
- Two factors affect utilization: the number of time slots used and the fraction of each time slot that is used for useful work.

- Depending on the deadlines for some of the processes, we may need to leave some time slots empty.

- Since the time slots are of equal size, some short processes may have time left over in their time slot

- Another scheduling policy that is slightly more sophisticated is ***round robin***. As in the figure, round robin uses the same hyperperiod as does cyclostatic.

- It also evaluates the processes in order.



- But unlike cyclostatic scheduling, if a process does not have any useful work to do, the round-robin scheduler moves on to the next process in order to fill the time slot with useful work.

- In this example, all three processes execute during the first hyperperiod, but during the second one, *P*1 has no useful work and is skipped.

- The processes are always evaluated in the same order.

- The last time slot in the hyperperiod is left empty; if we have occasional, non-periodic tasks without deadlines, we can execute them in these empty time slots.

- Round-robin scheduling is often used in hardware such as buses because it is very simple to implement but it provides some amount of flexibility.

- In addition to utilization, we must also consider **scheduling overhead**—the execution time required to choose the next execution process, which is incurred in addition to any context switching overhead.

- In general, the more sophisticated the scheduling policy, the more CPU time it takes during system operation to implement it.

- Moreover, we generally achieve higher theoretical CPU utilization by applying more complex scheduling policies with higher overheads.

- The final decision on a scheduling policy must take into account both theoretical utilization and practical scheduling overhead.

MULTIPROCESSOR

- A ***multiprocessor*** is, in general, any computer system with two or more processors coupled together. Multiprocessors used for scientific or business applications tend to have regular architectures: several identical processors that can access a uniform memory space.

- We use the term ***processing element (PE)*** to mean any unit responsible for computation, whether it is programmable or not.

- Embedded system designers must take a more general view of the nature of multiprocessors. As we will see, embedded computing systems are built on top of an astonishing array of different multiprocessor architectures.

- The first reason for using an embedded multiprocessor is that they offer significantly better cost/performance—that is, performance and functionality per dollar spent on the system—than would be had by spending the same amount of money on a uniprocessor system.

- The basic reason for this is that processing element purchase price is a *nonlinear* function of performance.

- The cost of a microprocessor increases greatly as the clock speed increases.

- We would expect this trend as a normal consequence of VLSI fabrication and market economics. Clock speeds are normally distributed by normal variations in VLSI processes; because the fastest chips are rare, they naturally command a high price in the marketplace.

- Because the fastest processors are very costly, splitting the application so that it can be performed on several smaller processors is usually much cheaper.

- Even with the added costs of assembling those components, the total system comes out to be less expensive.

- Of course, splitting the application across multiple processors does entail higher engineering costs and lead times, which must be factored into the project.

- In addition to reducing costs, using multiple processors can also help with real time performance.

- We can often meet deadlines and be responsive to interaction much more easily when we put those time-critical processes on separate processors. Given that scheduling multiple processes on a single

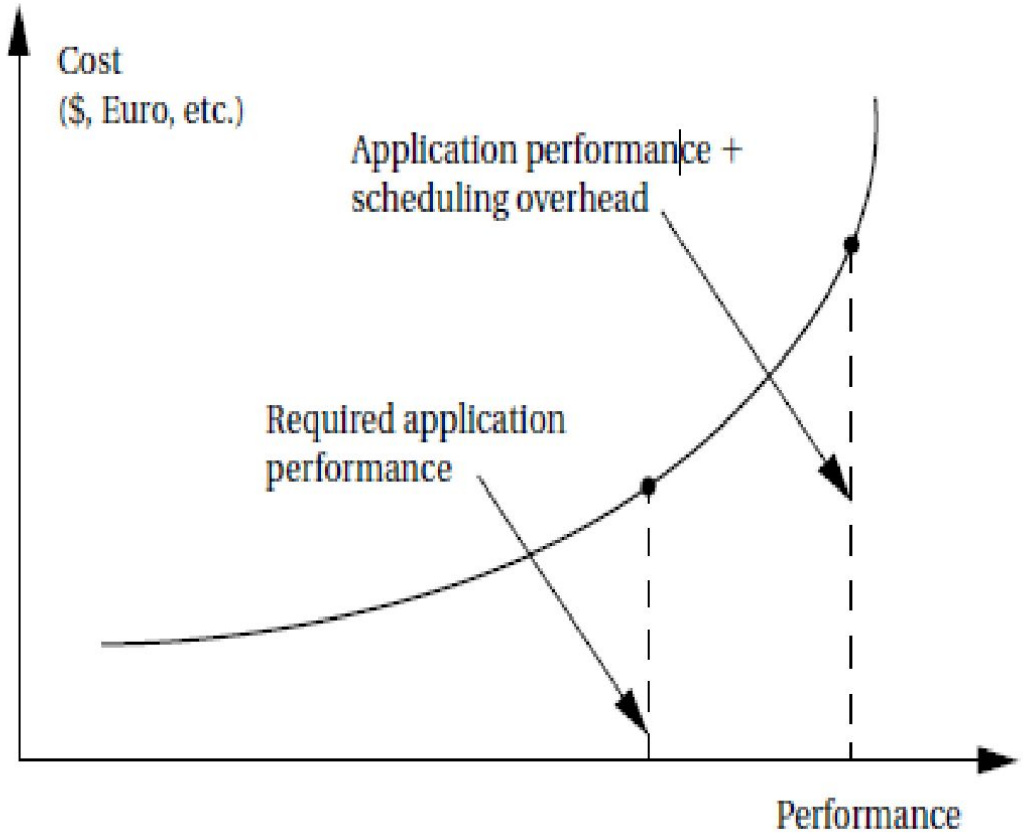
- Because we pay for that overhead at the nonlinear rate for the processor, as illustrated in the next figure, the savings by segregating time-critical processes can be large - it may take an extremely large and powerful CPU to provide the same responsiveness that can be had from a distributed system.

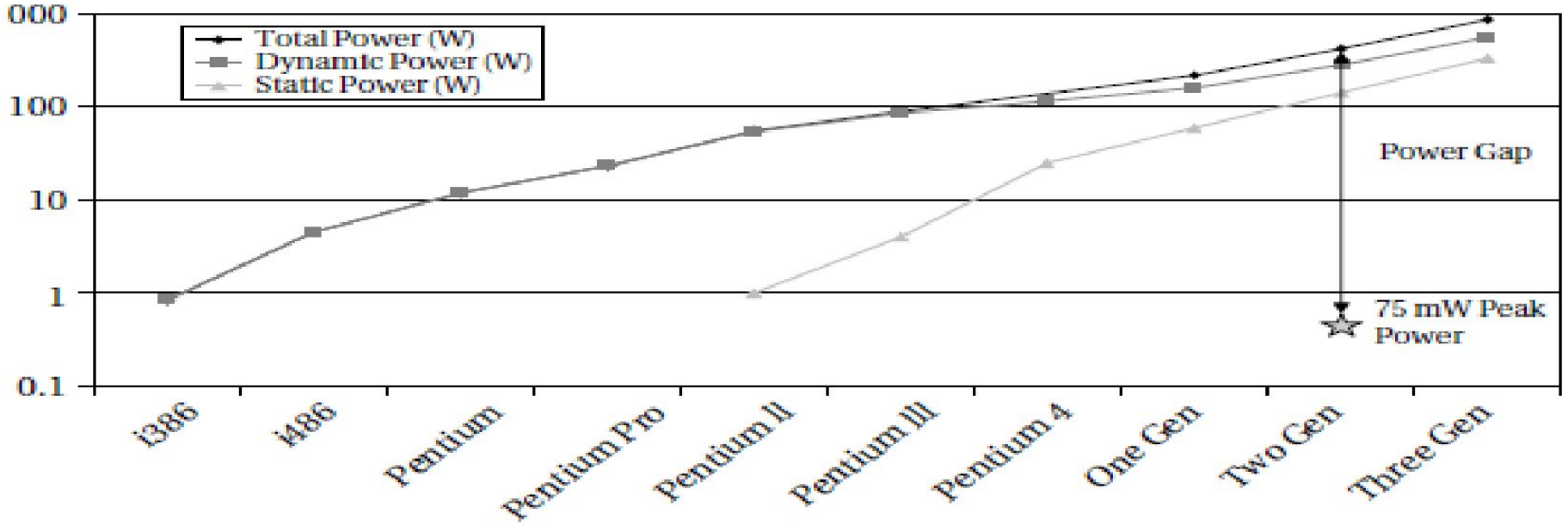
- Many of the technology trends that encourage us to use multiprocessors for performance also lead us to multiprocessing for low power embedded computing.

- Several processors running at slower clock rates consume less power than a single large processor: performance scales linearly with power supply voltage but power scales with V2.

- The general-purpose computing platforms are not keeping up with the strict energy budgets of battery-powered embedded computing.

Scheduling overhead is paid for at a nonlinear rate





Power consumption trends for desktop processors

- Here it compares the performance of power requirements of desktop processors with available battery power. Batteries can provide only about 75 Mw of power.

- Desktop processors require close to 1000 times that amount of power to run.

- That huge gap cannot be solved by tweaking processor architectures or software.

- Multiprocessors provide a way to break through this power barrier and build substantially more efficient embedded computing platforms.

THANK
YOU